

Exceptional Control Flow

Sistemas de Computadores

Departamento de Engenharia Informática

Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

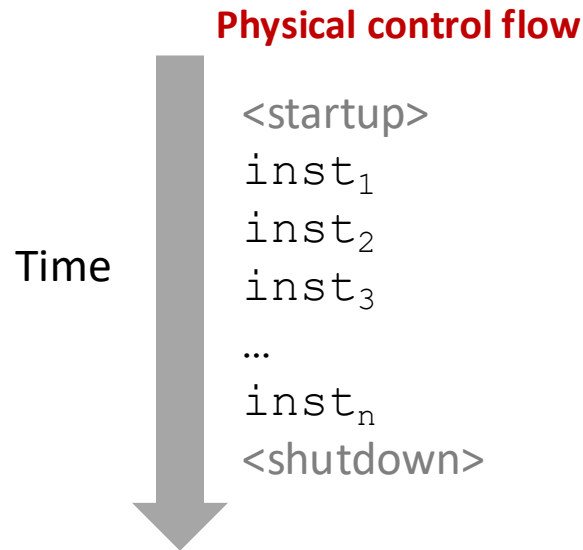
Overview

- **Exception control flow**
- **Synchronous exceptions**
- **Asynchronous exceptions**
- **Signals**

Control flow

■ Processors do only one thing at a time

- From startup to shutdown, each CPU core simply reads and executes a sequence of machine instructions, one at a time *
- This sequence is the CPU's control flow



- * many modern CPUs execute several instructions at once and/or out of program order, but this is invisible to the programmer

Altering the control flow

■ In ARQCP, we saw two mechanisms for changing control flow:

- Jumps and branches
- Call and return
- (react to changes in **program state**)

■ Insufficient for a useful system

- Difficult to react to changes in **system state**
- E.g., data arrives from a disk or a network adapter, instruction divides by zero, user hits Ctrl-C at the keyboard, system timer expires

Altering the control flow

■ Systems need mechanisms for exceptional control flow

- ECF is a change in control flow triggered by a system event (not by the program's normal instruction sequence)

■ Why ECF is needed

- Fix abnormal conditions (e.g., page faults, division by zero)
- Handle notifications from external devices (e.g., disk, network, timer)
- Support preemptive scheduling (timer interrupts)
- Provide a secure interface between applications and the OS (system calls)

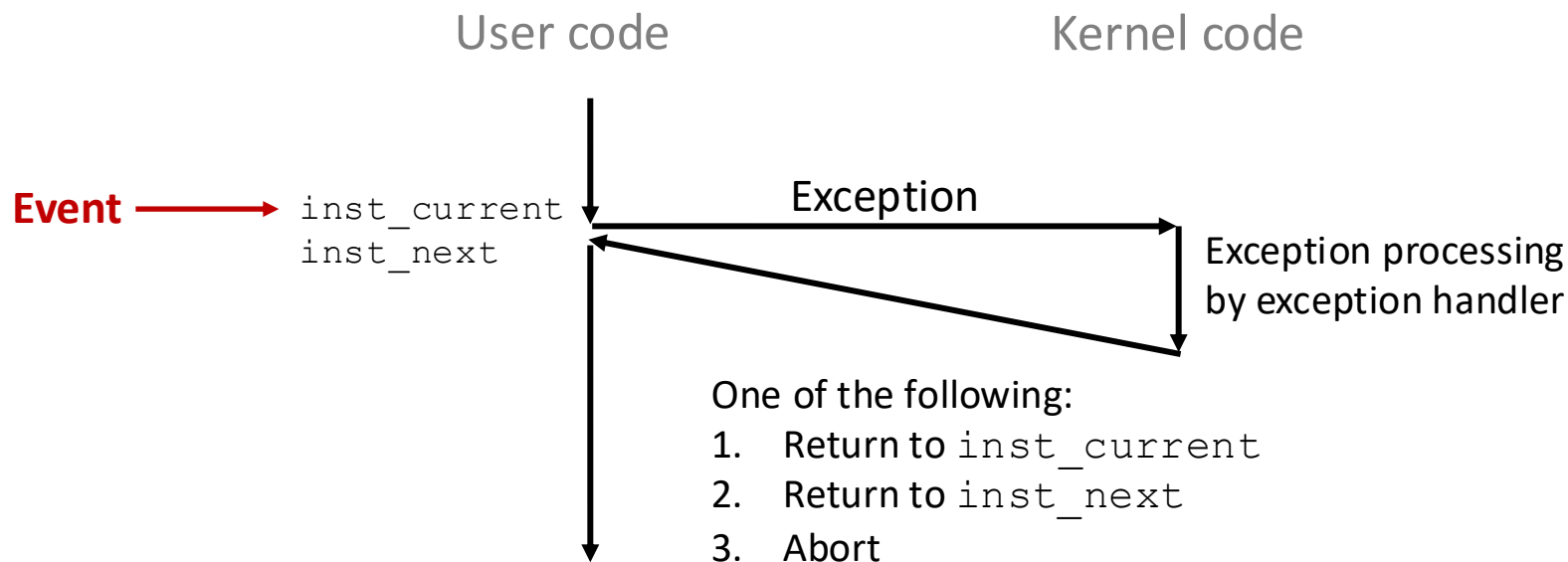
■ ECF exists at all levels of the system

- Hardware interfaces are explicitly designed to support the OS

Kernel-level exceptions

■ After OS booting, **all entry to kernel is a result of some event**

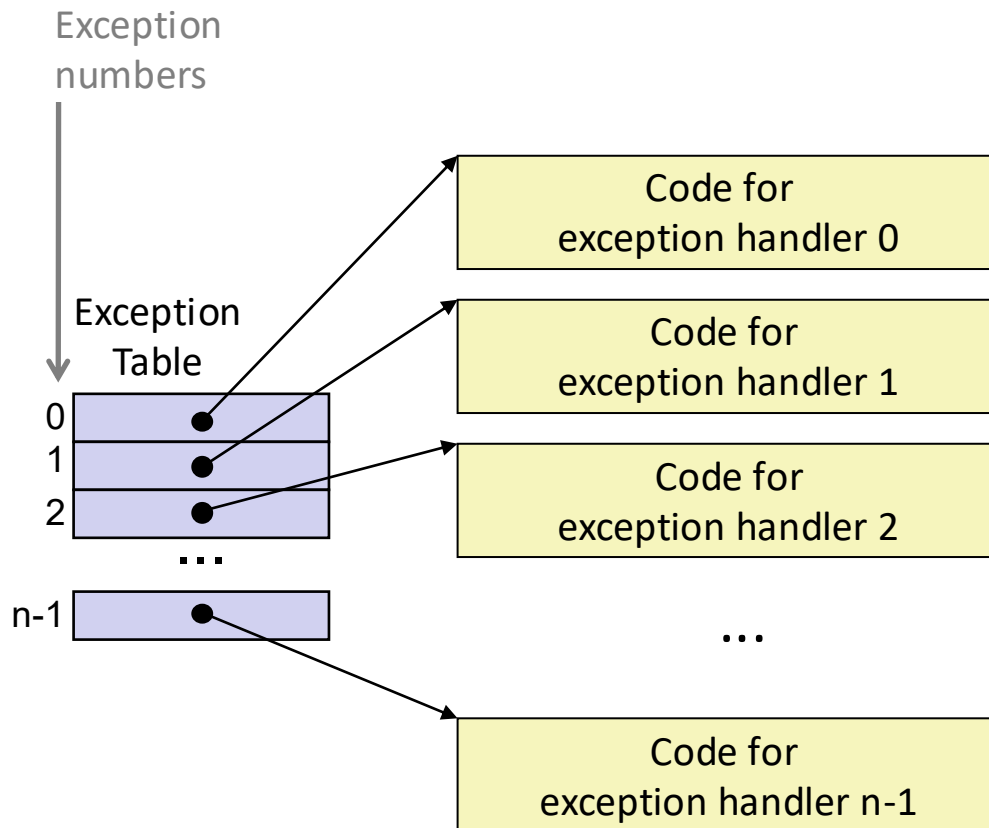
- Event immediately stops current execution
- Changes mode to kernel mode
- Invoke a piece of code to handle event (event handler)



Exception tables

- **During OS initialization, the kernel provides a handler to be invoked by the CPU when the exception occurs**
 - When the exception occurs, the CPU can change the current protection level (e.g., switch from user mode to kernel mode)
- **The **Interrupt Vector Table** is used to associate interrupt requests with handlers**
 - Each entry specifies the address of the handler
 - Architecture-specific implementation

Exception tables



- Each type of event has a unique exception number k
- k = index into exception table (a.k.a. interrupt vector)
- Handler k is called each time exception k occurs

Exceptions in Linux/x86-64

■ There are up to 256 different exception types

- 0-31 correspond to exceptions that are defined by the Intel architects and thus are identical for any x86-64 system
- 32-255 correspond to interrupts and traps that are defined by the OS
- Historically, systems calls are provided through exception 128 (0x80)
- Modern x86-64 systems use the `syscall` instruction instead

Number	Description	Exception class
0	Divide error	Fault
13	General protection fault	Fault
14	Page fault	Fault
18	Machine check	Abort
32-255	OS-defined exceptions	Interrupt or trap

Exceptional control flow: Today

■ Synchronous exceptions

- Change in control flow caused directly by executing an instruction
- Implemented using combination of hardware and OS software

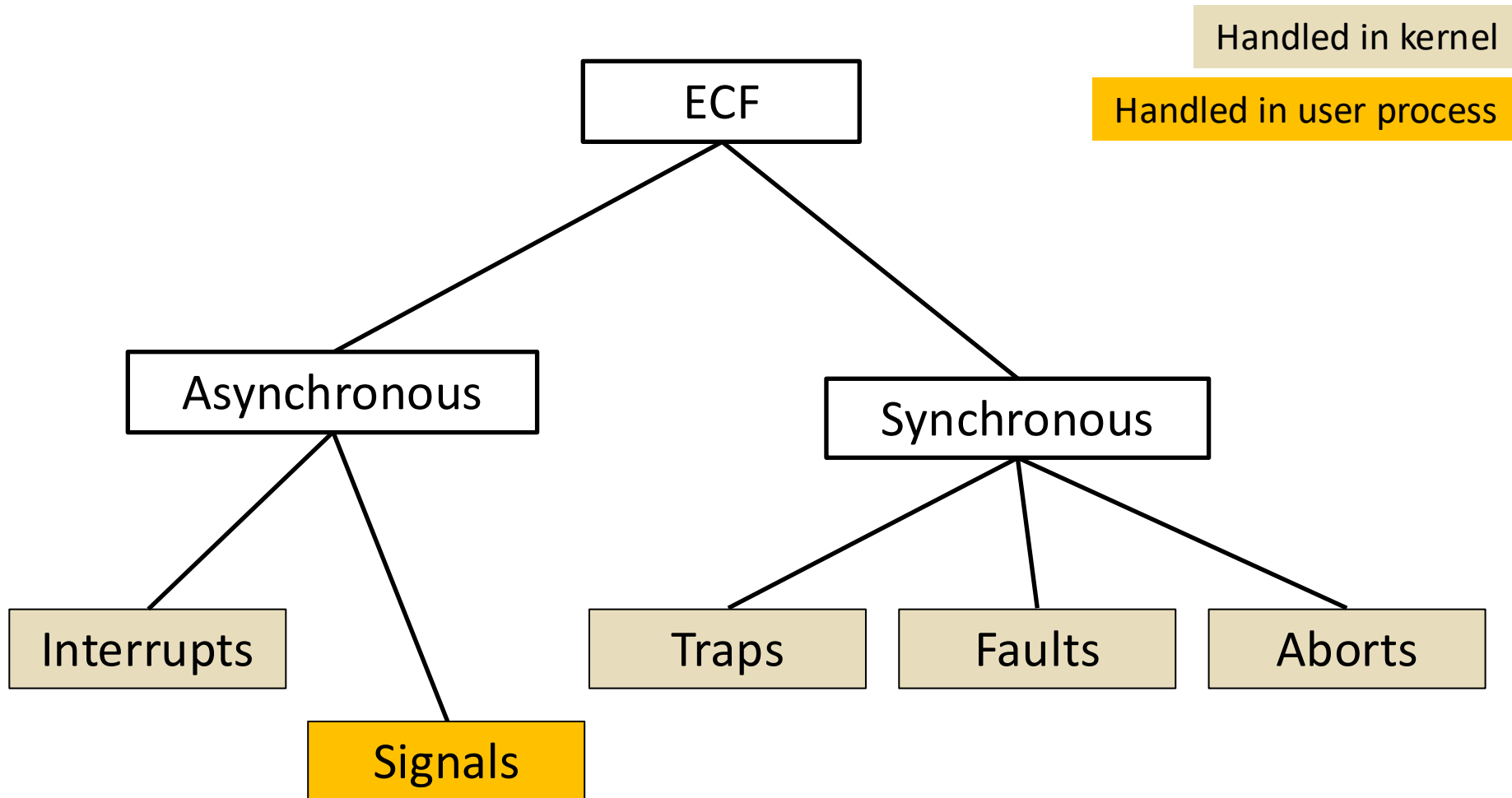
■ Process context switching (asynchronous exceptions)

- OS removes the executing job from the CPU and allocates it to another one
- Implemented by OS software and a hardware timer

■ Signals

- A small message that notifies a process that an event of some type has occurred in the system
- Implemented by OS and application software

ECF taxonomy



Overview

- Exception control flow
- **Synchronous exceptions**
- Asynchronous exceptions
- Signals

Synchronous exceptions

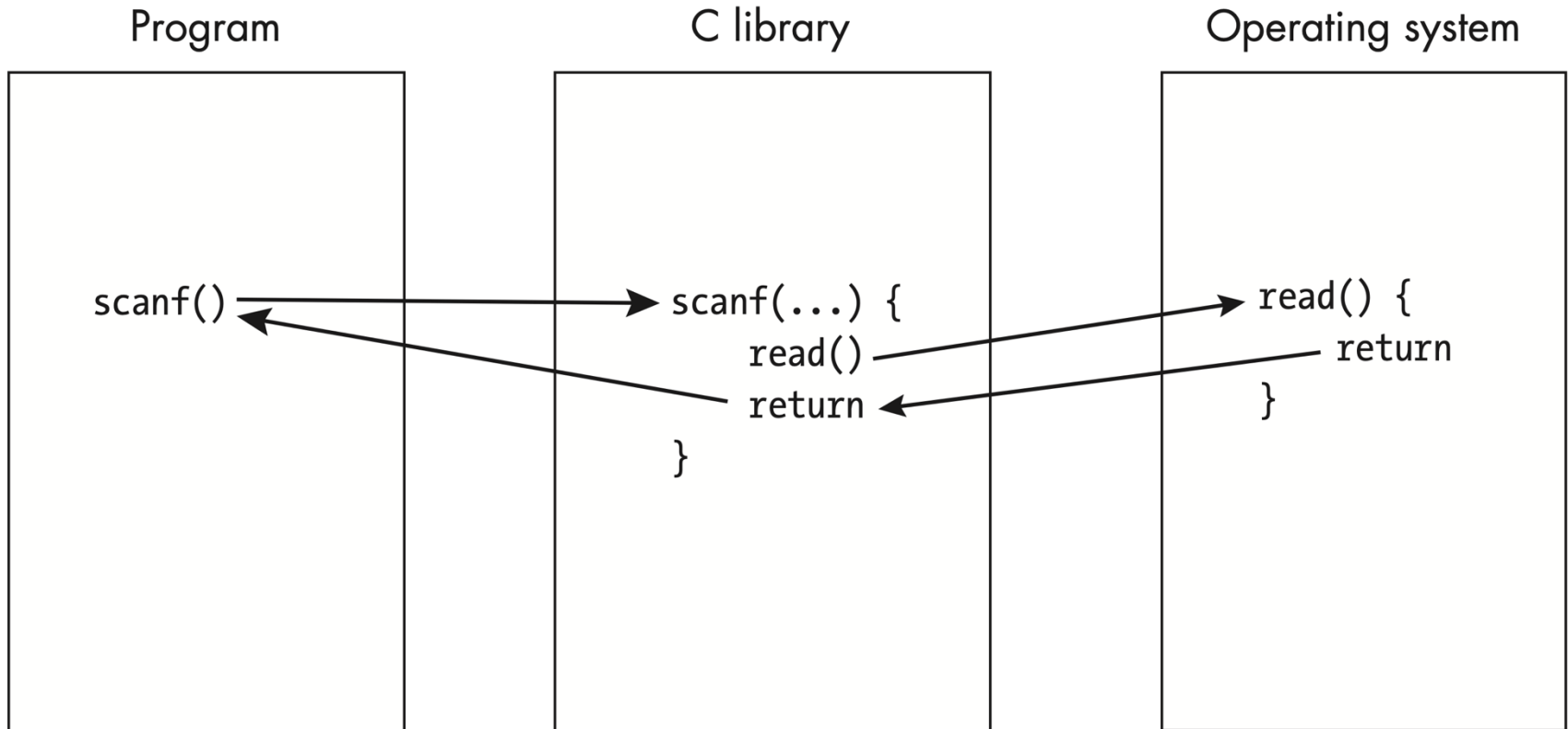
- **Occur directly as a consequence of executing an instruction**
 - E.g., dividing by zero or accessing invalid memory
 - They are deterministic events tied to the program's own operations, rather than external or asynchronous triggers
- **A **trap** is an intentional software-generated exception**
 - Set program up to “trip the trap” and do something
 - E.g., system calls, `gdb` breakpoints
- **Returns control to **next** instruction**

System calls

- **The primary mechanism through which user programs request services from the OS kernel**
 - Invoked via a special instruction (a “trap”) that triggers an exception
 - This controlled exception transfers execution from user mode to kernel mode

- **Acts as a bridge between user applications and privileged OS operations**
 - Ensures that critical operations are executed securely within the kernel
 - Provides a safe and structured interface for performing sensitive operations while maintaining system stability and security

System calls



System calls in Linux/x86-64

- Each Linux/x86-64 system call has a unique ID number

- Follows the x86-64 System V ABI

- Examples:

Number	Name	Description
0	read	Read file
1	write	Write file
2	open	Open file
3	close	Close file
4	stat	Get info about file
57	fork	Create process
59	execve	Execute a program
60	_exit	Terminate process
62	kill	Send signal to process

Trap example: System call

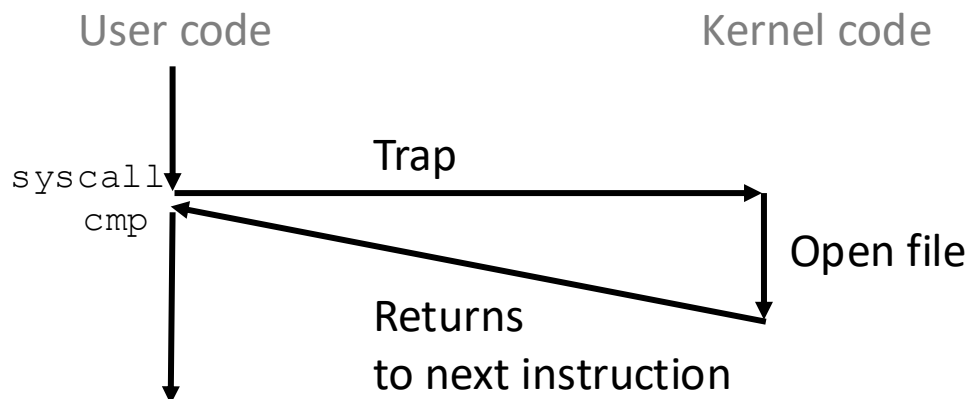
■ User calls `open(filename, options)`

- `open` calls `__open` function, which invokes system call instruction `syscall`

```

000000000000e5d70 <__open>:
...
e5d79:  b8 02 00 00 00      movq $0x2,%rax      # open is syscall #2
e5d7e:  0f 05              syscall             # Return value in %rax
e5d80:  48 3d 01 f0 ff ff   cmp  $0xffffffffffffffff001,%rax
...
e5dfa:  c3                retq

```



- `%rax` contains syscall number
- Other arguments in `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8`, `%r9`
- Return value in `%rax`
- Negative value is an error corresponding to negative `errno`

System calls

■ Almost like a function call

- Jumps to a specific routine (the kernel service) and then returns to the next instruction in user space
- Passes arguments using calling convention
- Gets result in `%rax`

■ Important differences!

- Executed in **kernel space**, which means they have complete access to all system resources
- “Address” of “function” is in `%rax`, **rather than a normal function pointer**
- Relies on `errno` to communicate errors
- Triggered by a trap (`syscall` on x86-64), not an ordinary `call`

Additional synchronous exceptions

- A **fault** is an unintentional, synchronous exception
 - Triggered directly by instruction execution
- Indicate a potentially recoverable error condition
 - Page fault — recoverable (page brought into memory)
 - Protection fault — unrecoverable (illegal access)
 - Floating-point exception — may be recoverable or fatal
- Control flow
 - If the OS can recover, **resume at the faulting instruction**
 - Otherwise, **terminate the process**

Fault example: Page fault

■ User writes to memory location

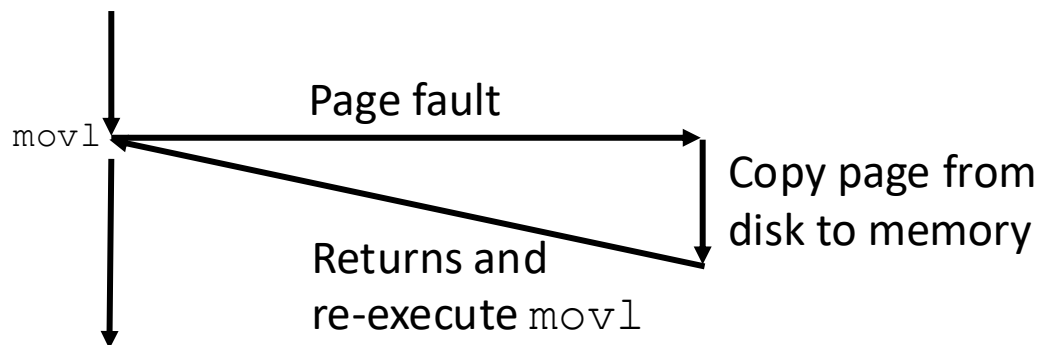
- That portion (page) of user's memory is **currently on disk**

```
int a[1000];  
...  
a[500] = 13;
```

```
80483b7:      c7 05 10 9d 04 08 0d  movl    $0xd, 0x8049d10
```

User code

Kernel code



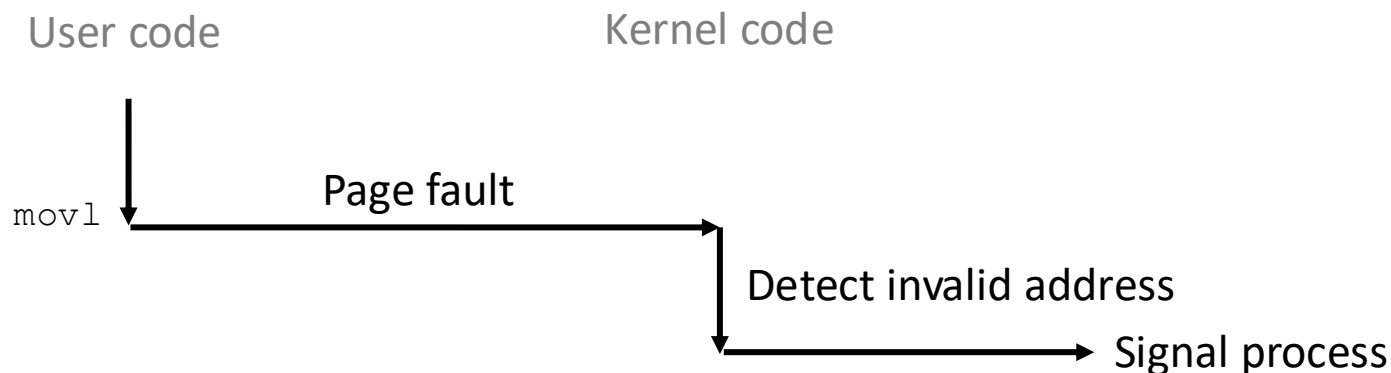
Fault example: Invalid memory reference

■ User writes to memory location

- That portion (page) of user's memory is **invalid**

```
int a[1000];
...
a[5000] = 13;
```

```
80483b7:      c7 05 60 e3 04 08 0d  movl    $0xd,0x804e360
```



■ Sends SIGSEGV signal to user process

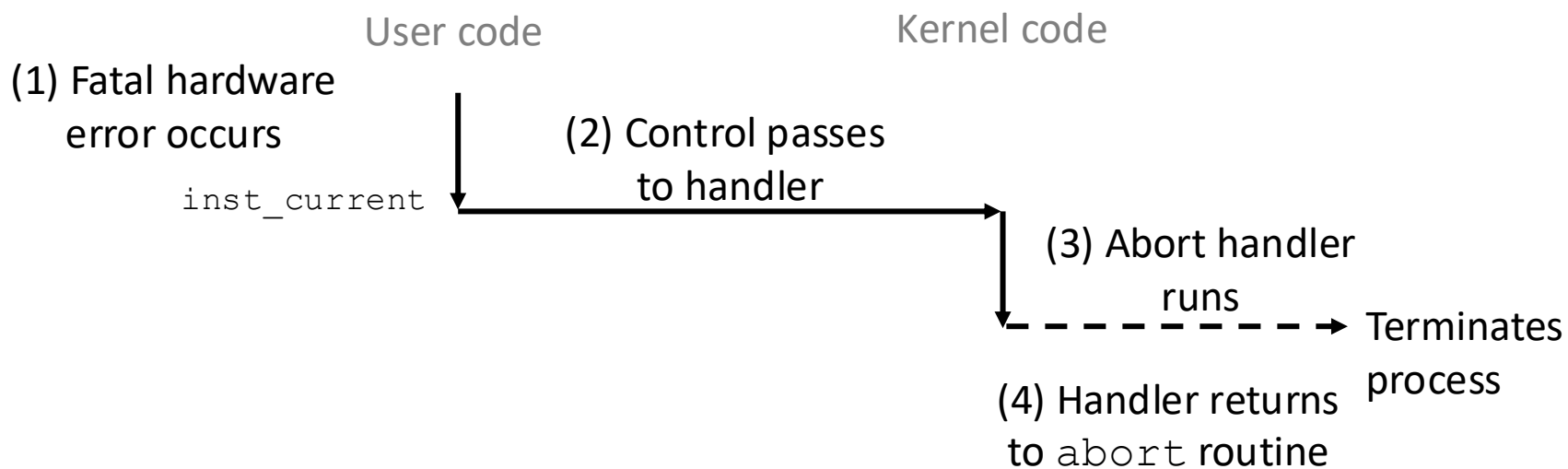
- User process exits with “segmentation fault”

Additional synchronous exceptions

- **Aborts** are nonrecoverable hardware failures that force an immediate halt in execution
 - No safe recovery is possible
- **Indicate severe system-level failures**
 - Such as memory parity errors, system bus faults, or cache malfunctions
- **Execution cannot resume from the interrupted operation**
 - The OS terminates the affected process (and may halt or reboot the system)

Abort example

- **Abort handlers never return control to the application program**
 - The handler returns control to an `abort` routine that terminates the application program



Overview

- Exception control flow
- Synchronous exceptions
- **Asynchronous exceptions**
- Signals

Asynchronous exceptions

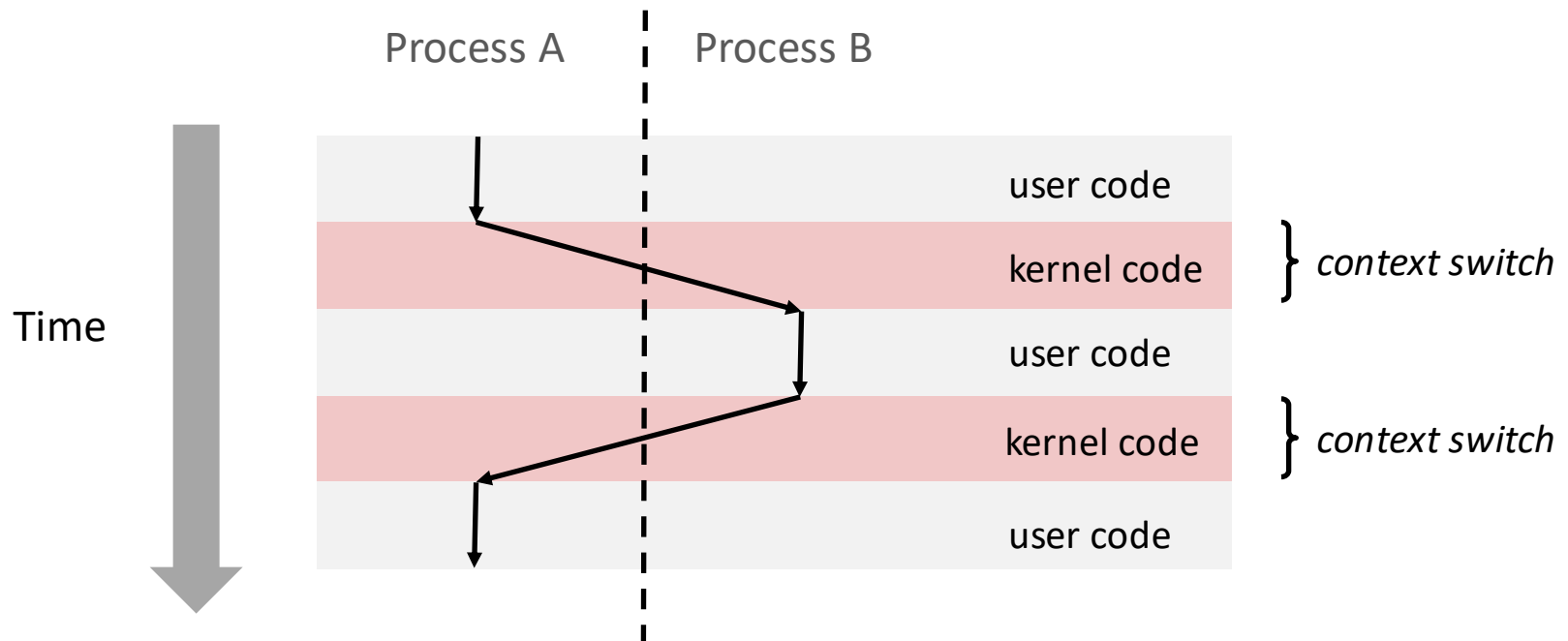
- **Occur independently of the current instruction execution**
 - Also called **asynchronous interrupts**
- **Can occur at any moment, regardless of the instruction flow**
- **The handler runs asynchronously, ensuring prompt response to critical events**
 - CPU pauses the current task and transfers control to an interrupt handler or signal handler

Asynchronous exceptions

- **Asynchronous exceptions enable the OS to handle external events promptly**
 - Ensuring that even while executing regular code, the system remains responsive and can manage concurrent activities effectively
- **E.g., timer interrupt**
 - Every few ms, an external timer chip triggers an interrupt
 - Used by the kernel to take back control from user programs
- **E.g., I/O interrupt from external device**
 - Hitting Ctrl-C at the keyboard
 - Arrival of a packet from a network
 - Arrival of data from a disk

Context switching

- For processes to run “as if they were on their own”, we need a routine for the **context switch**
 - That saves and subsequently restores all required state necessary for running a task

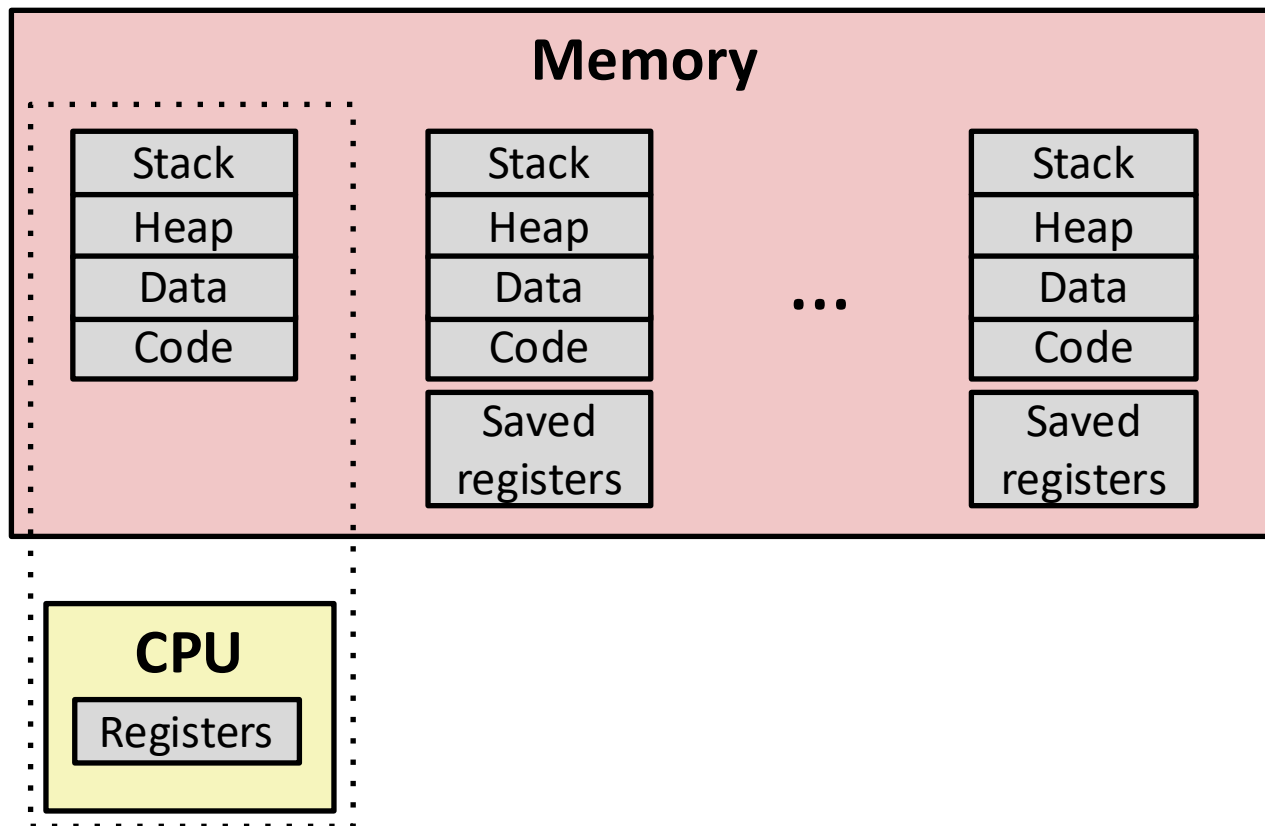


Multitasking

- **There are two fundamental concepts to build multitasking on:**
 - In **cooperative multitasking**, each process releases the CPU voluntarily by calling the context switch directly at suitable places
 - In **preemptive multitasking**, the context switch is triggered periodically by a timer interrupt

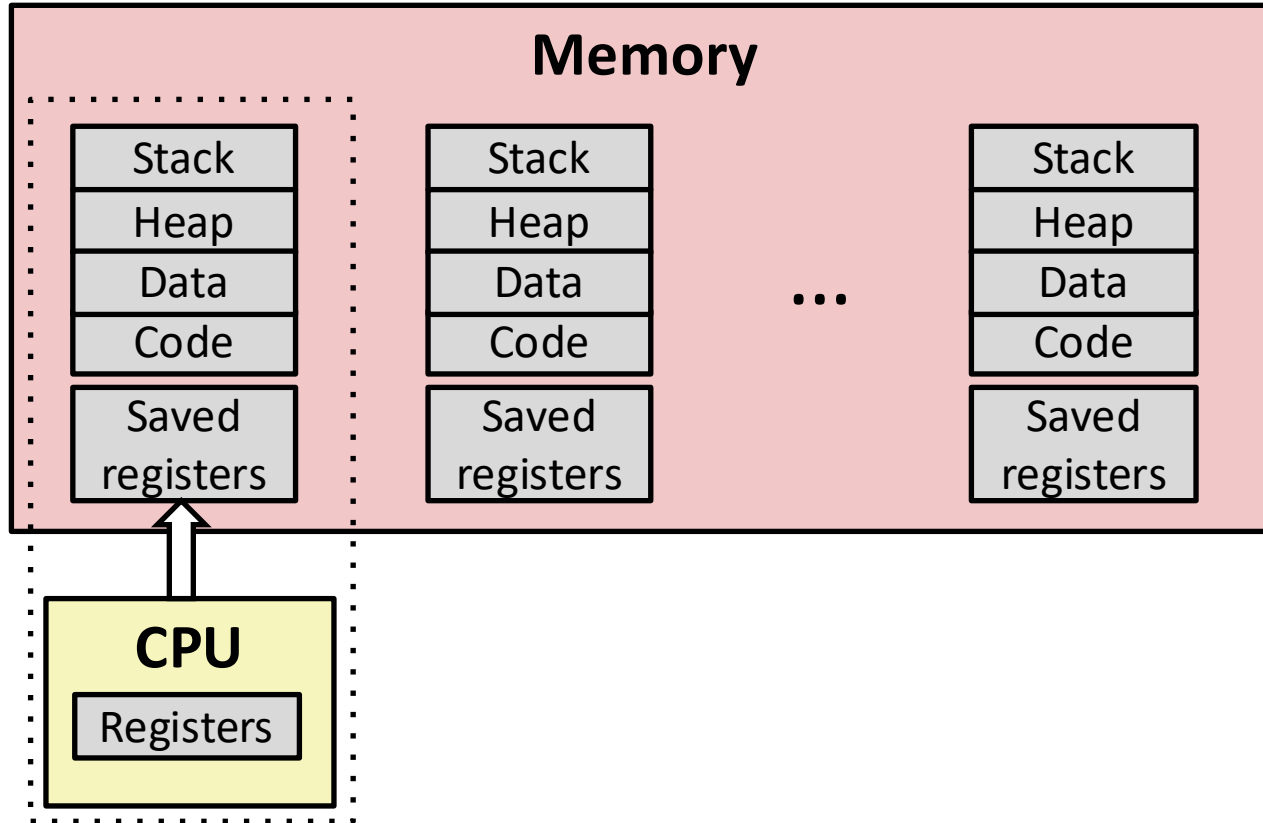
- **OS handles the timer interrupt by:**
 1. Deciding which process should run next (scheduling)
 2. Performing a context-switch to another process, then starting the timer again

Multitasking



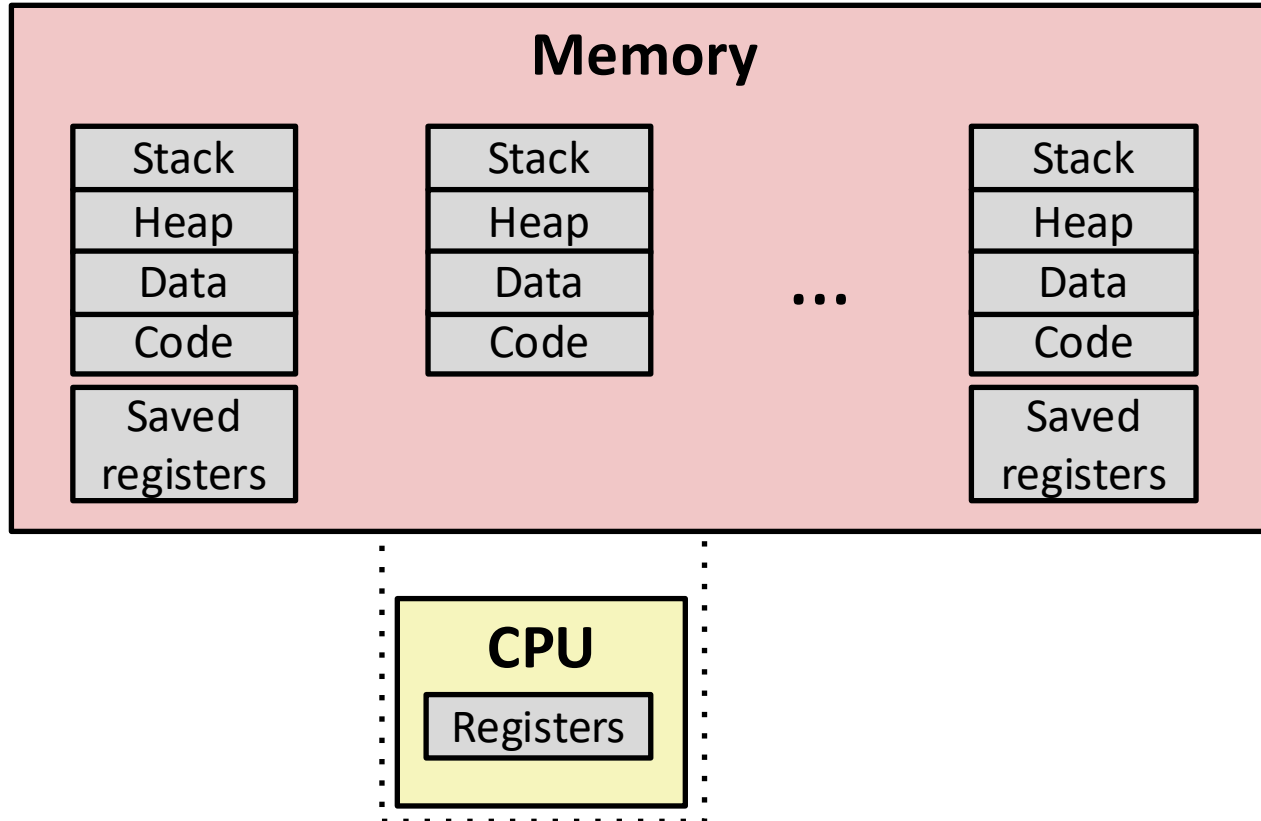
- **Single processor executes multiple processes concurrently**
 - Process executions interleaved (multitasking)
 - Address spaces managed by virtual memory system (like last week)
 - Register values for non-executing processes saved in memory

Multitasking



- Save current registers in memory

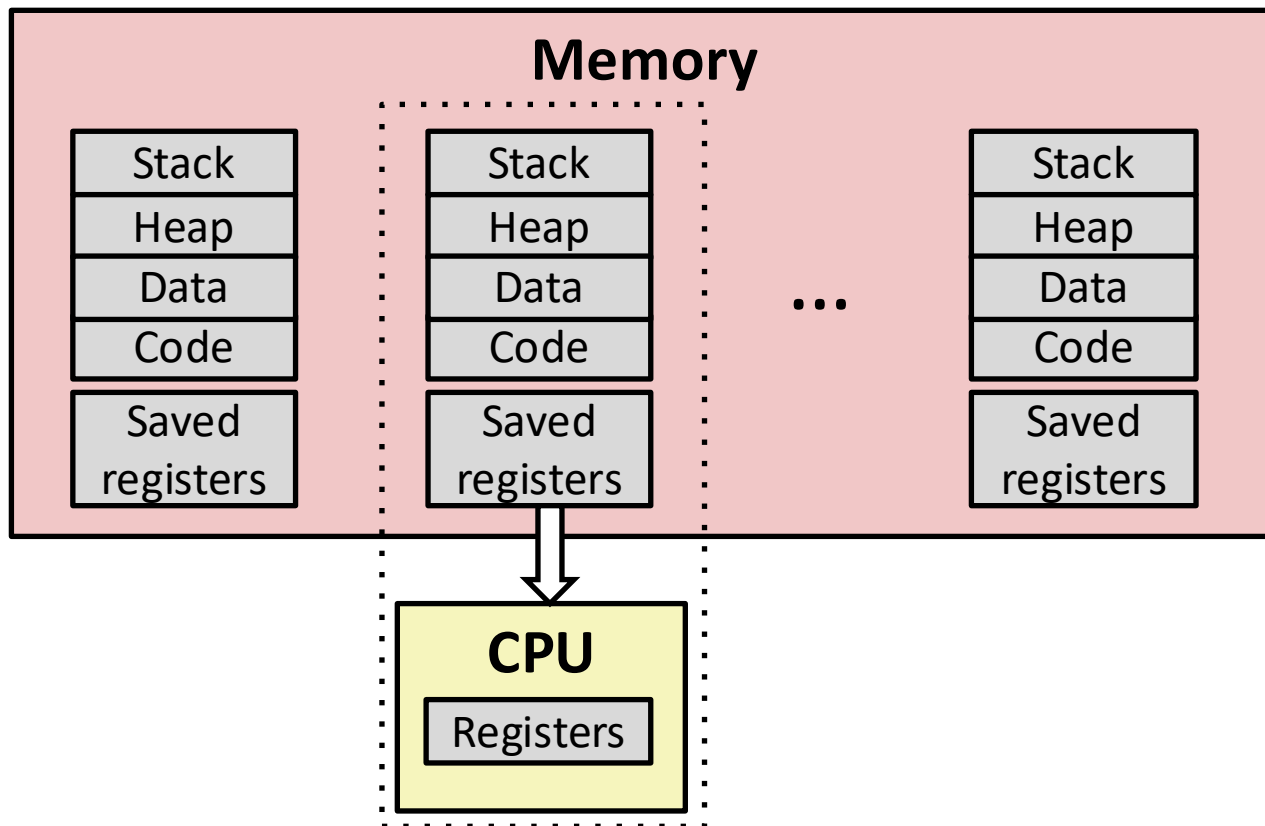
Multitasking



■ Schedule next process for execution

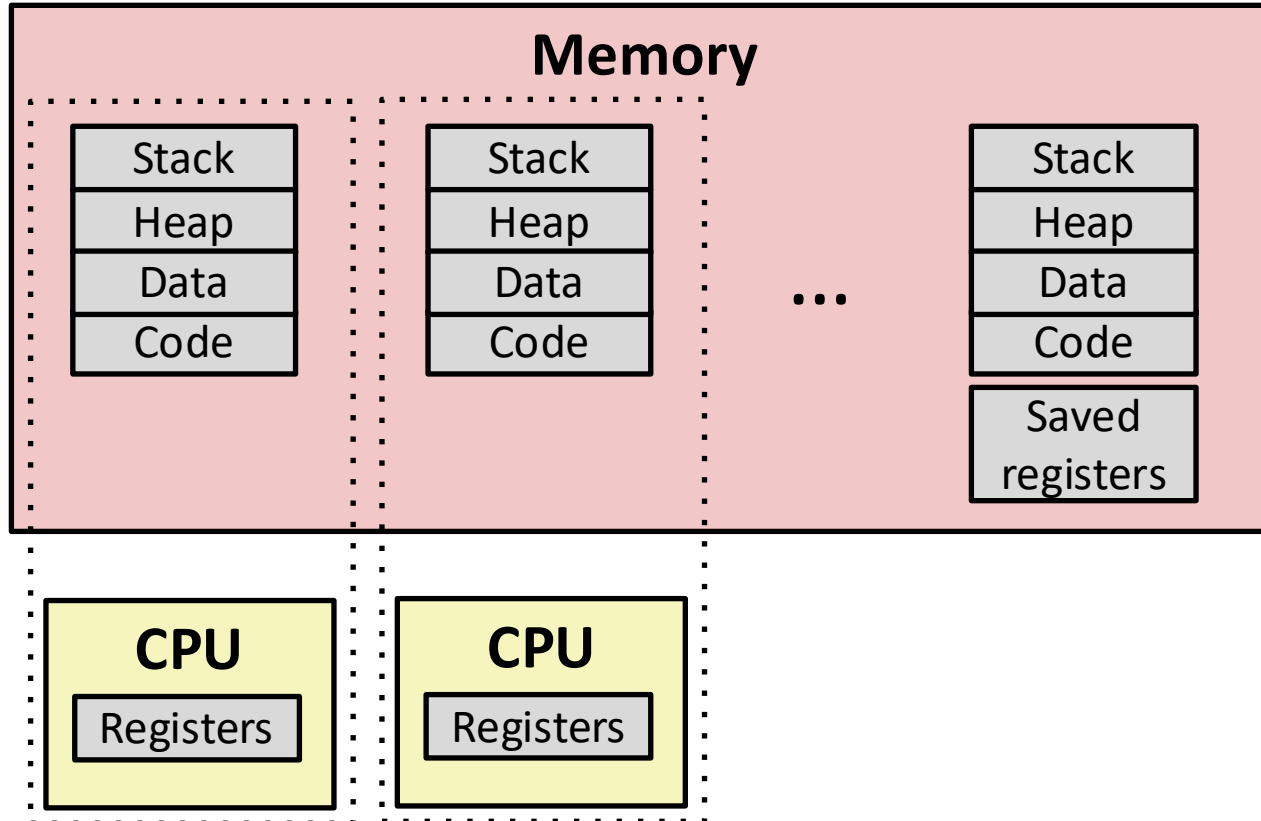
- Process scheduling is a large and complex topic
- (will spend more time on it in future lectures)

Multitasking



- Load saved registers and switch address space (**context switch**)
 - The kernel maintains a context for each process, the state that the kernel needs to restart a preempted process

Multitasking (multicore)



■ Multicore processors

- Multiple CPUs on single chip
- Share main memory (and some caches)
- Each can execute a separate process

Process Control Block

- Each process is represented in the OS by a **Process Control Block (PCB)**
 - Serves as the repository for all the data needed to start, or restart, a process, along with some accounting data

ID	IDType	<i>Identification</i>	
CPU State	StateType	State Vector Page Table Flags ... Unit Flags ...	
Processor	Int		
Memory	● →		
Resources	● →		
Status	StatusType	Running, Ready, Blocked To process' current queue	
Status Data	● →		
Parent	● →	Parent process List of children	
Children	● →		
Priority	Int	<i>Other</i>	
	...		

PCB in Linux

■ In Linux, the PCB is represented by the C `task_struct` structure

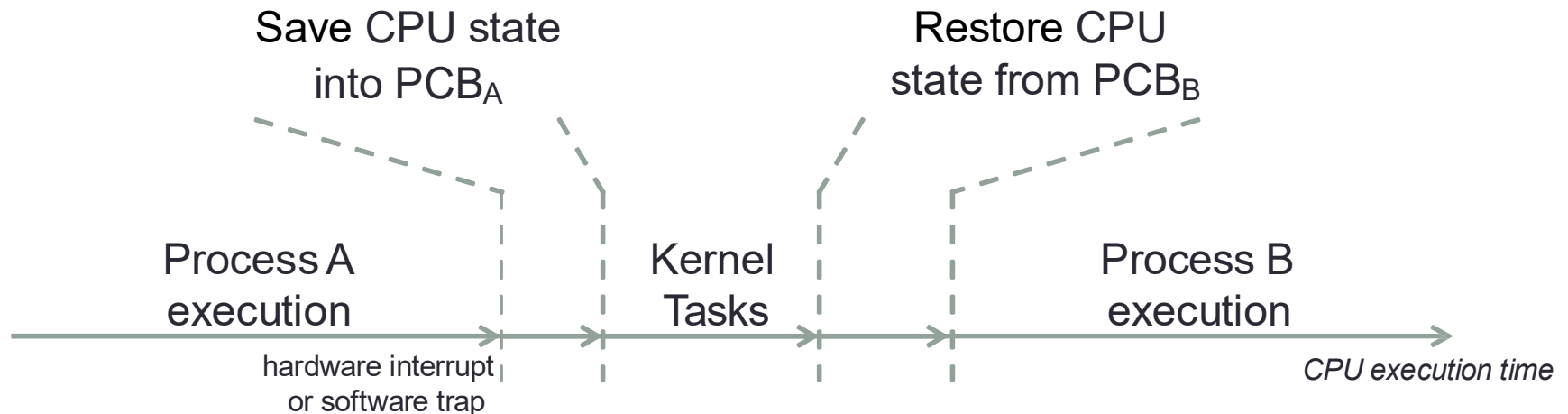
- Found in `<include/linux/sched.h>` in the kernel source-code directory

■ Some of its fields:

- `long state;` `/* state of the process */`
- `struct sched_entity se;` `/* scheduling information */`
- `struct task_struct *parent;` `/* this process's parent */`
- `struct list_head children;` `/* this process's children */`
- `struct files_struct *files;` `/* list of open files */`
- `struct mm_struct *mm;` `/* address space */`

Context switching

- When the OS switches from running a given process, the **process' context must be saved** into the process' PCB
 - CPU state: registers, program counter, stack pointer, status flags
 - (Other process state is usually already recorded in the PCB)
- Similarly, when the OS switches to another process, the new **process' context must be restored** from the PCB

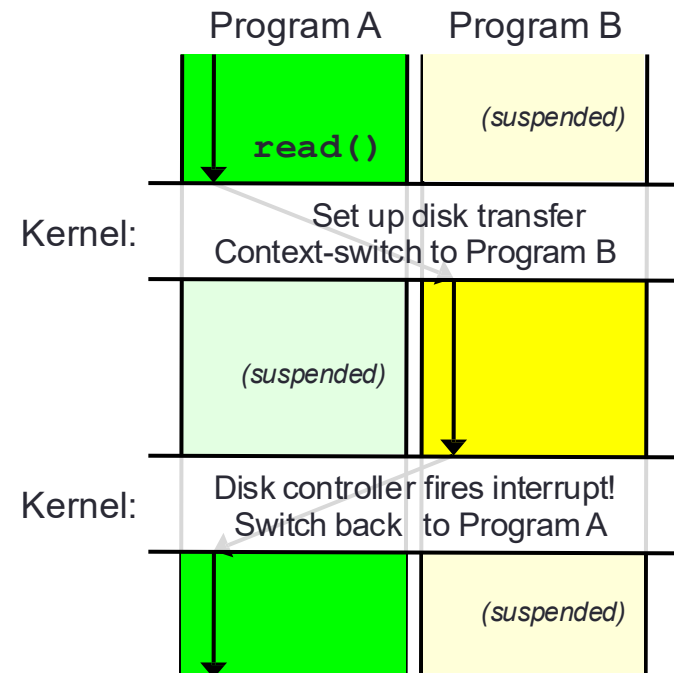


Context switching

- **During a context-switch, the OS isn't doing useful work**
 - By “useful work,” we mean “running the user's applications”
- **Want to minimize amount of time a context-switch takes**
 - e.g., make the scheduler fast, save/load CPU state fast, etc.
 - (Still, context-switches will take some amount of time...)
- **Also want to minimize the frequency of context-switches**
 - If our system performs many context-switches, it will be spending less time doing useful work
- **Will cover more details when we get to scheduling**

Long running I/O operations

- Hardware interrupt handling is also very important with long running I/O operations (and most of them are)
 - e.g., reading a block from a magnetic disk can take 3-15ms
 - e.g., reading a block from an SSD can take several μ s
- When a process performs a long running I/O operation:
 - The process is unable to proceed until the I/O operation completes
 - OS sets up the I/O operation, then **switches to a process that can run**
 - When the I/O operation is finished, the hardware fires an interrupt to notify the OS



Long running I/O operations

■ Key idea

- Interrupts allow the OS to react asynchronously to external events without wasting CPU time in busy waiting

■ The interrupt handler:

- Identifies which device generated the interrupt
- Retrieves the result of the I/O operation
- Updates the state of the process waiting for the I/O

■ Result:

- The previously blocked process becomes ready to run
- The scheduler may select it again for execution

Overview

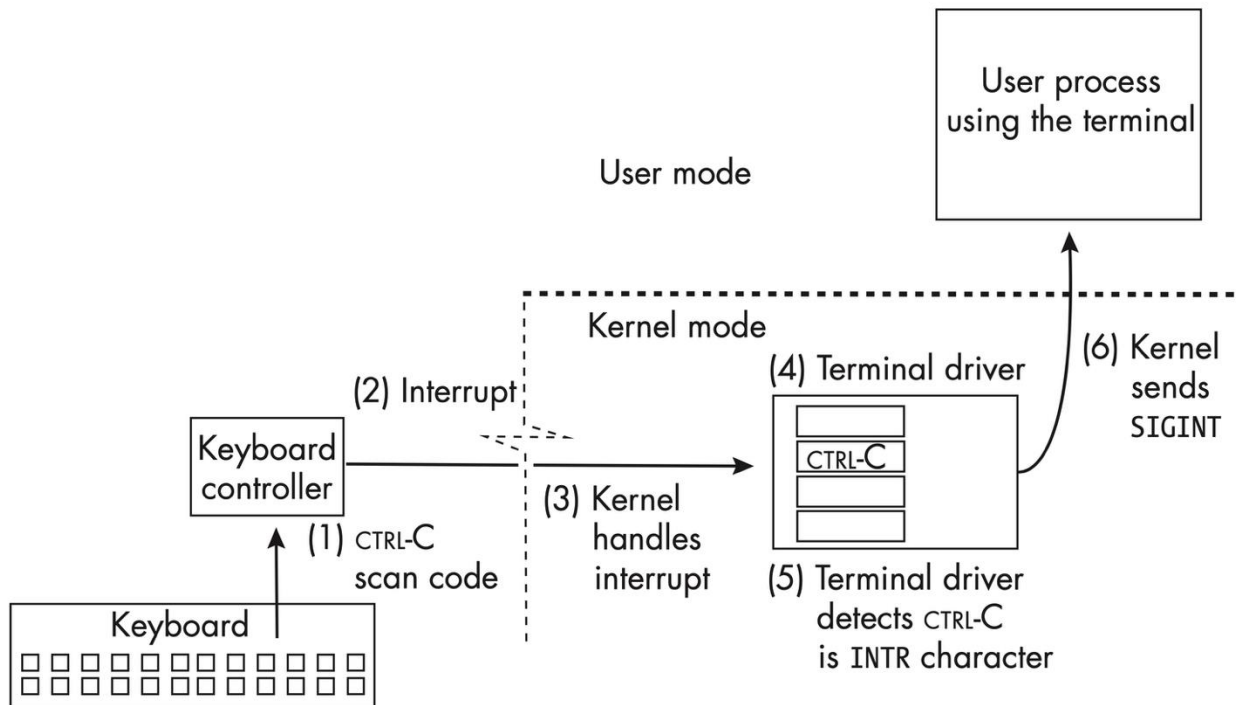
- Exception control flow
- Synchronous exceptions
- Asynchronous exceptions
- **Signals**

Signals

- **Signals are an OS-level abstraction built on top of exceptions and interrupts**
 - Not a hardware exception class
- **Are *software interrupts* that inform a process of an event requiring immediate attention**
 - Offer a lightweight mechanism for exceptional control flow, essential for responsive, robust concurrent programs
- **They serve as *asynchronous triggers*, redirecting execution from the normal flow to a signal handler or default action**
 - Signals can arrive anytime, requiring careful synchronization with ongoing tasks

Signals

- **Regardless of the source of the event, it is only the kernel that sends signals to processes**
 - Signal type is identified by small integer ID's (1-30)
 - A signal has no other information associated with it besides its value



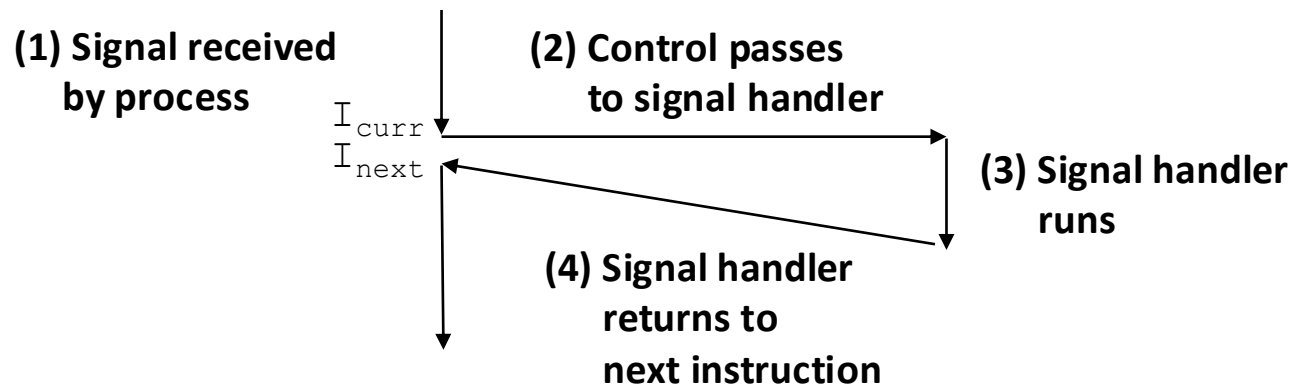
Signals

- **The Linux kernel provides a diverse set of standard and real-time signals, each associated with a default action**
 - Such as terminating the process, generating a core dump, or simply being ignored, which defines how the OS handles specific asynchronous events

ID	Name	Default Action	Corresponding Event
2	SIGINT	Terminate	User typed ctrl-c
9	SIGKILL	Terminate	Kill program (cannot override or ignore)
11	SIGSEGV	Terminate	Segmentation violation
14	SIGALRM	Terminate	Timer signal
17	SIGCHLD	Ignore	Child stopped or terminated

Receiving a signal

- A destination process **receives** a signal when it is forced by the kernel to react in some way to the signal
- Some possible ways to react:
 - **Ignore** the signal (do nothing)
 - **Terminate** the process (with optional core dump)
 - **Catch** the signal by executing a user-level function called signal handler



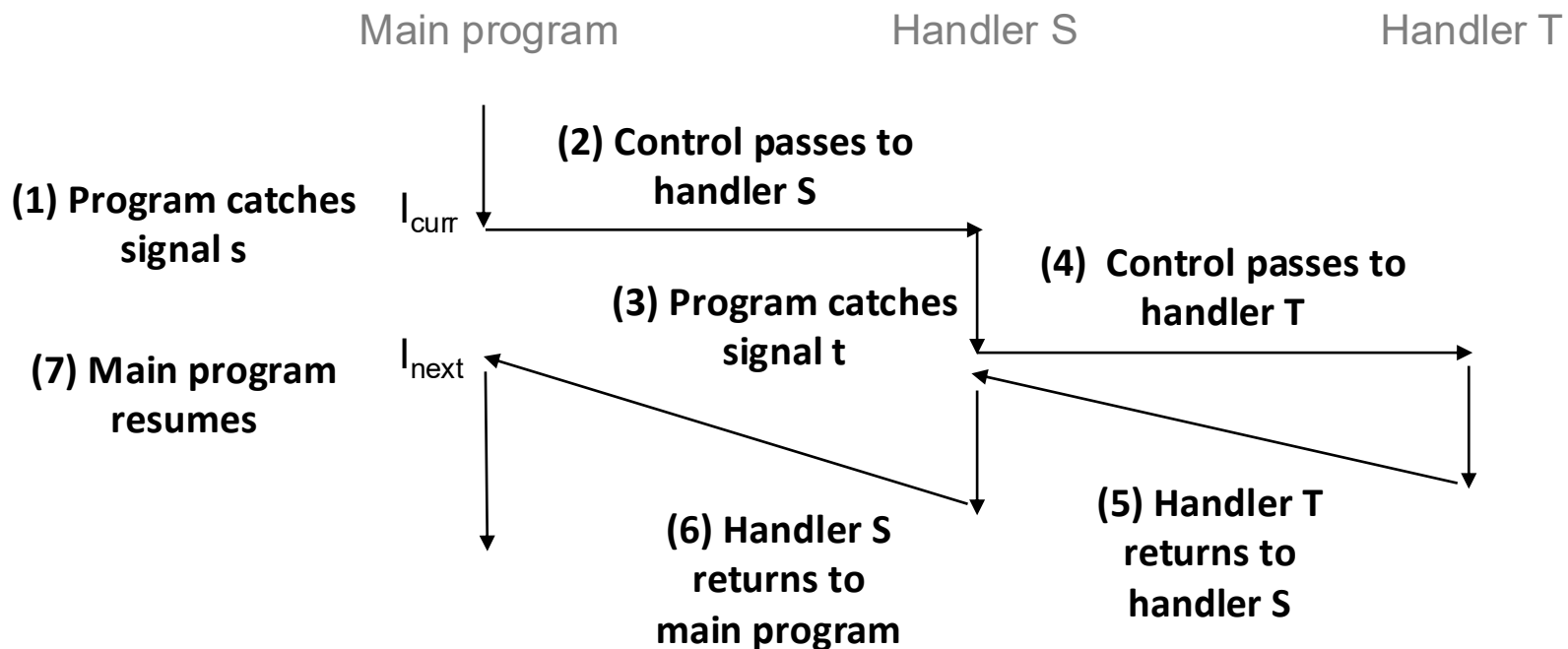
Handling a signal reception

- **Handlers are custom functions that override default behaviors**
 - Should be minimal and avoid unsafe calls to prevent race conditions or deadlocks
 - Only async-signal-safe functions may be called inside a signal handler
- **When you design a program, if you create and register a signal handler for a specific signal, you've set its disposition**
 - By not registering a signal handler, you've also set its disposition to accept the default action

Nested signal handlers

■ Handlers can be interrupted by other handlers

- Adds complexity to signal handling logic to avoid issues such as race conditions or deadlocks (discussed in future lectures)



Summary

- **Exceptions are events that require nonstandard control flow**
- **Can be generated externally (interrupts) or internally (traps, faults and aborts)**
 - Synchronous exceptions directly result from executing an instruction (e.g., system calls, page faults, division by zero, invalid memory access)
 - Asynchronous Exceptions: Occur independently of the current instruction (e.g., hardware interrupts, signals)
- **Provide a structured mechanism for error handling, system calls, and inter-process communication while preserving system stability and security**